

МИНИСТЕРСТВО ОБРАЗОВАНИЯ КИРОВСКОЙ ОБЛАСТИ
Кировское областное государственное профессиональное образовательное
бюджетное учреждение
«Слободской колледж педагогики и социальных отношений»

ОТЧЕТ

по производственной практике

ПМ.02. Осуществление интеграции программных модулей

Студент

Шишкин Артём Григорьевич

Группа 23П-1

Специальность 09.02.07

Информационные системы и
программирование

Руководитель практики от колледжа:

Седов А.С

Руководитель практики от организации:

_____/_____

подпись

УТВЕРЖДАЮ:

Директор

Наименование организации

_____/_____

подпись

расшифровка

М. П.

2025-2026 уч. Год

Содержание

Оглавление

1. Характеристика объекта практики.....	2
2. Описание рабочего места.....	4
3. Состав программного и технического обеспечения, имеющегося на предприятии, их назначение	5
4. Описание выполненных видов работ	7
__ 4.2. Проектирование ERD (Entity-Relationship Diagram)	8
__ 4.3. Проектирование базы данных	10
__ 4.4. Импорт данных в БД	12
__ 4.5. Разработка программного модуля	13
__ 4.6. Интеграция с API.....	14
__ 4.7. Отладка программных модулей	16
__ 4.8. Доработка обработкой исключительных ситуаций	17
__ 4.9. Рефакторинг программного кода	18
__ 4.10. Ручное тестирование	19
__ 4.11. Модульное тестирование	20
__ 4.12. Интеграционное тестирование	21
__ 4.13. Фиксация результатов в системе контроля версий	22
__ 4.14. Разработка руководства оператора	23
__ 4.15. Разработка пояснительной записки	23
5. Заключение.....	24

1. Характеристика объекта практики

Объектом практики является КОГОВУ СШ пгт Вахруши – муниципальное общеобразовательное учреждение, осуществляющее образовательную деятельность по программам начального, основного и среднего общего образования.

Юридический адрес: 613110, Кировская область, Слободской район, пгт Вахруши, улица Ленина, дом 4.

Специализация учреждения: реализация общеобразовательных программ, организация учебного процесса, воспитательная работа, обеспечение информационной безопасности, ведение хозяйственной деятельности.

Структура учреждения: директор, заместители директора по учебной и воспитательной работе, педагогический состав (около 40 учителей), административно-хозяйственный персонал, системный администратор (1 человек), технический персонал. Всего в школе работает около 60 сотрудников.

В школе имеется значительное количество компьютерного и офисного оборудования, требующее систематического учёта: компьютерные классы (3 кабинета), интерактивные доски (5 штук), проекторы (10 штук), принтеры и МФУ (15 штук), серверное оборудование, сетевое оборудование, ноутбуки для учителей.

2. Описание рабочего места

Практика проходила на рабочем компьютере в школе, а также на личном ноутбуке для разработки и тестирования.

Характеристика рабочего места в организации (Таблица 1):

Таблица 1 – Характеристика рабочего места

Компонент	Описание
Оборудование	Стационарный ПК (Intel Core i3-7100, 6 ГБ ОЗУ, встроенная видеокарта Intel HD Graphics 630)
Операционная система	Windows 10 Pro (лицензионная)
Средства разработки	Microsoft Visual Studio 2022 Community (облегчённая конфигурация), SQL Server Management Studio
СУБД	Microsoft SQL Server 2019 Express Edition (бесплатная версия, оптимизирована для ограниченных ресурсов)
Дополнительное ПО	Git, Postman, .NET SDK 8.0/10.0

Особенности работы в условиях ограниченных ресурсов:

В связи с тем, что компьютер в организации имел ограниченные характеристики (6 ГБ ОЗУ и слабая встроенная видеокарта), были приняты следующие меры для обеспечения комфортной работы:

Выбор лёгких сред разработки: использование Visual Studio 2022 Community с отключёнными тяжёлыми расширениями (ReSharper, CodeRush), работа в режиме "Без отладки" (Ctrl+F5) для экономии памяти.

Использование SQL Server Express: выбрана лёгкая версия СУБД, потребляющая минимальное количество оперативной памяти.

Разработка через удалённое подключение: часть разработки выполнялась на личном ноутбуке с более высокими характеристиками (16 ГБ ОЗУ) с последующим переносом кода через Git.

Оптимизация Visual Studio: отключены фоновые анализаторы кода, уменьшен размер кэша IntelliSense, использовался режим "Без открытия обозревателя решений" для ускорения загрузки.

3. Состав программного и технического обеспечения, имеющегося на предприятии, их назначение

В школе используется следующее программное и техническое обеспечение:

Программное обеспечение (Таблица 2):

Таблица 2 – Программное обеспечение школы

ПО	Назначение
Windows 10 Pro (лицензионная)	Операционная система для рабочих станций
Windows Server 2019	Серверная ОС для файлового сервера и сервера баз данных
Microsoft Office 365	Офисный пакет для работы с документами
1С:Школьный учёт	Учёт успеваемости и кадров
Антивирус Kaspersky	Защита от вредоносного ПО
Mozilla Firefox / Chrome	Веб-браузеры для доступа к образовательным ресурсам
СЭД «Дело»	Система электронного документооборота

Техническое обеспечение (Таблица 3):

Таблица 3 – Техническое обеспечение школы

Оборудование	Количество	Назначение
Компьютеры (Intel Core i3/i5, 6-8 ГБ ОЗУ)	45 шт.	Рабочие станции для учителей и администрации
Ноутбуки	15 шт.	Мобильные рабочие места для учителей
Сервер HP ProLiant	1 шт.	Файловый сервер и сервер БД
Интерактивные доски	5 шт.	Оснащение кабинетов
Проекторы	10 шт.	Оснащение кабинетов
Принтеры и МФУ	15 шт.	Печать документов
Коммутаторы и сетевое оборудование	8 шт.	Локальная сеть, доступ в Интернет
Точки доступа Wi-Fi	4 шт.	Беспроводной доступ

4. Описание выполненных видов работ

4.1. Анализ предметной области

Предметной областью является процесс учёта компьютерного и офисного оборудования в школе. Оборудование распределено по кабинетам и зданиям, имеет различные характеристики, статусы и сроки службы.

Основные бизнес-процессы включают:

- поступление нового оборудования в школу (покупка, получение по гранту, передача);
- распределение оборудования по кабинетам и зданиям;
- изменение состояния оборудования (ремонт, списание, передача);
- контроль сроков гарантии и планирование замены;
- формирование отчётности о наличии и состоянии оборудования.

Выделены следующие категории пользователей: администратор (полный доступ), оператор (ведёт учёт оборудования), просмотрщик (только просмотр данных).

Диаграмма вариантов использования (Use Case) представлена на Рисунке 1.



Рисунок 1 – Диаграмма вариантов использования (Use Case)

4.2. Проектирование ERD (Entity-Relationship Diagram)

Логическая модель данных представлена в виде ER-диаграммы на Рисунке 2.

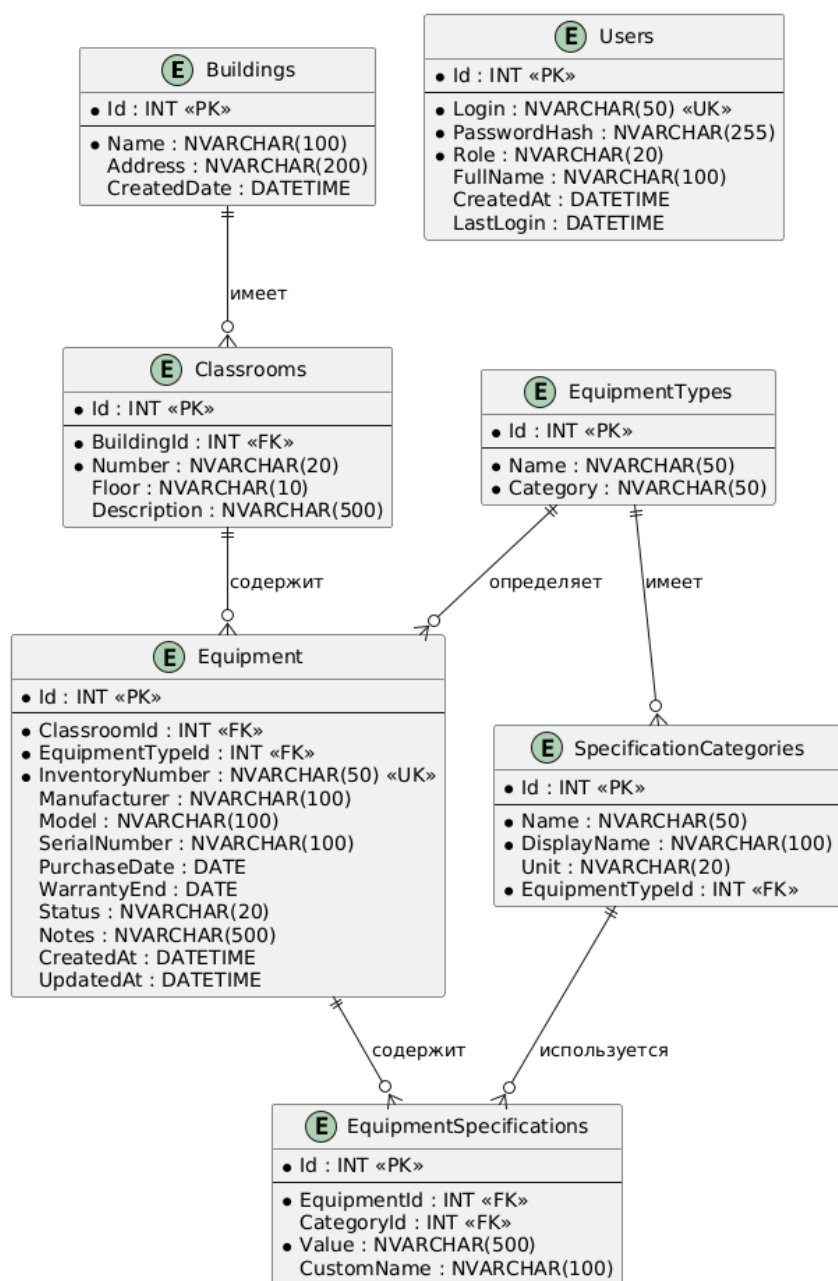


Рисунок 2 – ER-диаграмма базы данных

Актеры и их функциональные возможности (Таблица 4):

Таблица 4 – Актеры и их функциональные возможности

Актор	Функциональные возможности
Администратор	Полный CRUD всех сущностей, управление пользователями, резервное копирование
Оператор	Создание, чтение, обновление оборудования; не может удалять
Просмотрщик	Только чтение всех данных

4.3. Проектирование базы данных

На основе ER-модели разработана реляционная база данных, нормализованная до третьей нормальной формы (3НФ). Определены следующие таблицы (Таблица 5):

Таблица 5 – Структура базы данных

№	Таблица	Описание
1	Buildings	Здания школы (название, адрес)
2	Classrooms	Кабинеты (привязка к зданию, номер, этаж, описание)
3	EquipmentTypes	Типы оборудования (название, категория)
4	Equipment	Оборудование (инвентарный номер, производитель, модель, серийный номер, дата покупки, окончание гарантии, статус, примечания)
5	SpecificationCategories	Категории характеристик (код, отображаемое имя, единица измерения, привязка к типу)
6	EquipmentSpecifications	Характеристики оборудования (значение, пользовательское имя)
7	Users	Пользователи системы (логин, хэш пароля, роль, ФИО)

Связи между таблицами реализованы через внешние ключи с каскадным удалением.

Физическая модель реализована в СУБД Microsoft SQL Server 2019 Express Edition.

Скрипт создания таблиц представлен в Листинге 1.

Листинг 1 – Скрипт создания таблиц (фрагмент)

```
CREATE TABLE Buildings (  
    Id INT IDENTITY(1,1) PRIMARY KEY,  
    Name NVARCHAR(100) NOT NULL,  
    Address NVARCHAR(200) NULL,  
    CreatedDate DATETIME DEFAULT GETDATE()  
);  
  
CREATE TABLE Classrooms (  
    Id INT IDENTITY(1,1) PRIMARY KEY,  
    BuildingId INT NOT NULL,  
    Number NVARCHAR(20) NOT NULL,  
    Floor NVARCHAR(10) NULL,  
    Description NVARCHAR(500) NULL,  
    CONSTRAINT FK_Classrooms_Buildings FOREIGN KEY (BuildingId) REFERENCES  
Buildings(Id) ON DELETE CASCADE  
);  
  
CREATE TABLE EquipmentTypes (  
    Id INT IDENTITY(1,1) PRIMARY KEY,  
    Name NVARCHAR(50) NOT NULL,  
    Category NVARCHAR(50) NOT NULL  
);  
  
CREATE TABLE Equipment (  
    Id INT IDENTITY(1,1) PRIMARY KEY,  
    ClassroomId INT NOT NULL,  
    EquipmentTypeId INT NOT NULL,  
    InventoryNumber NVARCHAR(50) NOT NULL UNIQUE,  
    Manufacturer NVARCHAR(100) NULL,  
    Model NVARCHAR(100) NULL,  
    SerialNumber NVARCHAR(100) NULL,  
    PurchaseDate DATE NULL,  
    WarrantyEnd DATE NULL,  
    Status NVARCHAR(20) DEFAULT 'Working',  
    Notes NVARCHAR(500) NULL,  
    CreatedAt DATETIME DEFAULT GETDATE(),  
    UpdatedAt DATETIME DEFAULT GETDATE(),  
    CONSTRAINT FK_Equipment_Classrooms FOREIGN KEY (ClassroomId)  
REFERENCES Classrooms(Id) ON DELETE CASCADE,  
    CONSTRAINT FK_Equipment_EquipmentTypes FOREIGN KEY (EquipmentTypeId)  
REFERENCES EquipmentTypes(Id)  
);
```

4.4. Импорт данных в БД

Для заполнения базы тестовыми данными разработан T-SQL скрипт, генерирующий более 1000 записей. Скрипт добавляет:

- справочные данные (здания, кабинеты, типы оборудования, категории характеристик);
- оборудование (1000+ единиц) со случайным распределением по кабинетам и типам;
- характеристики для части оборудования (500+ записей);
- пользователей системы (admin, operator, viewer).

Фрагмент скрипта генерации данных представлен в Листинге 2.

Листинг 2 – Скрипт генерации тестовых данных (фрагмент)

```
DECLARE @i INT = 1;

WHILE @i <= 1000
BEGIN
    INSERT INTO Equipment (
        ClassroomId,
        EquipmentTypeId,
        InventoryNumber,
        Manufacturer,
        Model,
        SerialNumber,
        PurchaseDate,
        WarrantyEnd,
        Status
    ) VALUES (
        1 + ABS(CHECKSUM(NEWID())) % 30,
        1 + ABS(CHECKSUM(NEWID())) % 21,
        'INV-' + RIGHT('000000' + CAST(@i AS NVARCHAR(6)), 6),
        (SELECT TOP 1 value FROM STRING_SPLIT('HP,Dell,Lenovo,Apple,ASUS', ','))
    ORDER BY NEWID()),
        (SELECT TOP 1 value FROM
    STRING_SPLIT('ProBook,Latitude,ThinkPad,MacBook,VivoBook', ',') ORDER BY
    NEWID()),
        'SN' + CAST(ABS(CHECKSUM(NEWID())) AS NVARCHAR(15)),
        DATEADD(DAY, -ABS(CHECKSUM(NEWID())) % 2920, '2026-01-01'),
        DATEADD(MONTH, 12 + ABS(CHECKSUM(NEWID())) % 24, GETDATE()),
        CASE WHEN ABS(CHECKSUM(NEWID())) % 100 < 80 THEN 'Working'
             WHEN ABS(CHECKSUM(NEWID())) % 100 < 95 THEN 'Repair'
```

Результат импорта данных (Таблица 6):

Таблица 6 – Количество импортированных записей

Таблица	Количество записей
Buildings	3
Classrooms	30
EquipmentTypes	21
Equipment	1000+
SpecificationCategories	45
EquipmentSpecifications	500+
Users	3

4.5. Разработка программного модуля

Программный комплекс состоит из двух частей: серверного Web API и клиентского настольного приложения.

Серверная часть реализована на [ASP.NET](#) Core 10. Предоставляет REST-эндпоинты для:

- аутентификации (JWT токены);
- управления зданиями, кабинетами, типами оборудования;
- управления оборудованием и его характеристиками.

Доступ к данным осуществляется через Entity Framework Core.

Клиентская часть – приложение WPF (.NET Framework 4.8) с архитектурой MVVM. Пользовательский интерфейс адаптируется под роль авторизованного пользователя. Взаимодействие с сервером инкапсулировано в сервисе ApiService.

Структура проекта API (Таблица 7):

Таблица 7 – Структура проекта API

Папка	Назначение	Количество файлов
Controllers	Контроллеры API (6 шт.)	6
Models	Модели данных (7 шт.)	7
Data	Контекст БД и инициализация	2
DTOs	Объекты передачи данных (8 шт.)	8
Services	Сервисы (AuthService)	1
Middleware	Промежуточное ПО (JwtMiddleware)	1

Структура клиентского приложения (Таблица 8):

Таблица 8 – Структура клиентского приложения

Папка	Назначение	Количество файлов
Views	Окна приложения (7 шт.)	7
ViewModels	Модели представления (7 шт.)	7
Models	Модели данных (7 шт.)	7
Services	Сервисы (ApiService, AuthService, DialogService)	3
Helpers	Вспомогательные классы (RelayCommand, ViewModelBase)	2

4.6. Интеграция с API

Интеграция выполнена путём замены прямого доступа к базе данных в клиентском приложении на вызовы REST API. Сервис ApiService формирует HTTP-запросы, добавляет авторизационные заголовки (Bearer Token), обрабатывает ответы.

Диаграмма взаимодействия компонентов представлена на Рисунке 3.

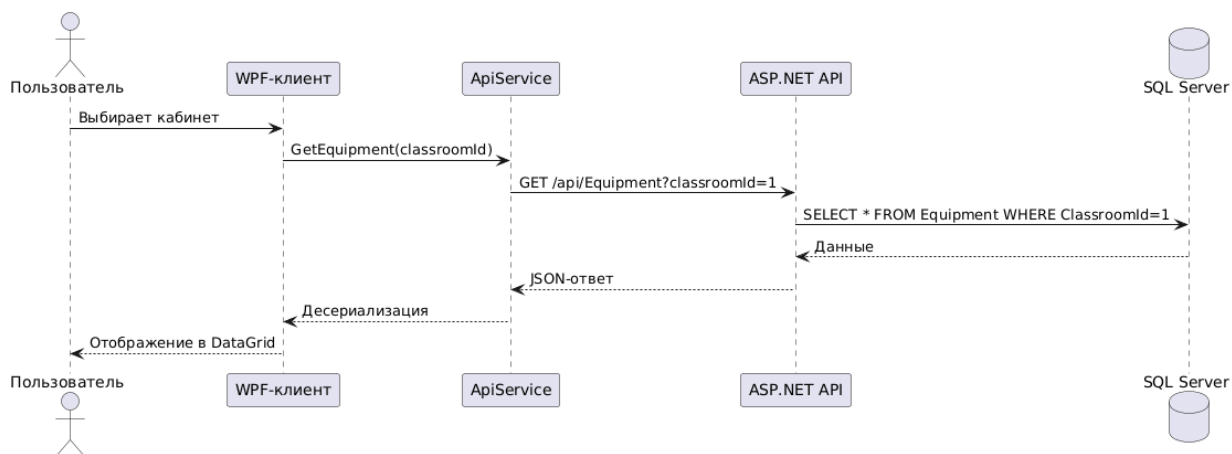


Рисунок 3 – Диаграмма взаимодействия клиента с API

Основные эндпоинты API (Таблица 9):

Таблица 9 – Основные эндпоинты API

№	Метод	URL	Описание	Доступ
1	POST	/api/Auth/login	Аутентификация	Все
2	GET	/api/Buildings	Получить все здания	Все
3	POST	/api/Buildings	Создать здание	Admin, Operator
4	PUT	/api/Buildings/{id}	Обновить здание	Admin, Operator
5	DELETE	/api/Buildings/{id}	Удалить здание	Admin
6	GET	/api/Classrooms	Получить все кабинеты	Все
7	POST	/api/Classrooms	Создать кабинет	Admin, Operator
8	PUT	/api/Classrooms/{id}	Обновить кабинет	Admin, Operator
9	DELETE	/api/Classrooms/{id}	Удалить кабинет	Admin
10	GET	/api/EquipmentTypes	Получить все типы	Все
11	POST	/api/EquipmentTypes	Создать тип	Admin, Operator
12	PUT	/api/EquipmentTypes/{id}	Обновить тип	Admin, Operator
13	DELETE	/api/EquipmentTypes/{id}	Удалить тип	Admin
14	GET	/api/Equipment	Получить оборудование	Все
15	GET	/api/Equipment/{id}	Получить оборудование по ID	Все

16	POST	/api/Equipment	Создать оборудование	Admin, Operator
17	PUT	/api/Equipment/{id}	Обновить оборудование	Admin, Operator
18	DELETE	/api/Equipment/{id}	Удалить оборудование	Admin

Пример взаимодействия клиента с API представлен в Листинге 6.

Листинг 6 – Пример вызова API из клиентского приложения

```
// В ApiService.cs
public async Task<List<Equipment>> GetEquipmentAsync(int? classroomId = null)
{
    var url = "Equipment?";
    if (classroomId.HasValue) url += $"classroomId={classroomId.Value}";
    return await GetAsync<List<Equipment>>(url);
}

// В EquipmentViewModel.cs
private async Task LoadEquipmentAsync(int classroomId)
{
    var equipment = await _api.GetEquipmentAsync(classroomId);
    Equipment.Clear();
    foreach (var e in equipment)
        Equipment.Add(e);
}
```

4.7. Отладка программных модулей

Отладка проводилась с использованием встроенных средств **Visual Studio 2022**, а также инструментов тестирования HTTP-запросов (**Postman**).

Основные этапы отладки:

1. Отладка серверной части:
 - Проверка корректности маршрутов и ответов
 - Проверка работы аутентификации (JWT)
 - Валидация входных данных
2. Отладка клиентской части:
 - Правильность формирования запросов
 - Обработка ответов сервера

- Корректность отображения данных в интерфейсе

3. Отладка интеграции:

- Проверка передачи данных между клиентом и сервером
- Проверка обработки ошибок

Выявленные и устранённые ошибки (Таблица 10):

Таблица 10 – Выявленные ошибки и их устранение

№	Ошибка	Решение
1	Циклические ссылки при сериализации JSON	Проецирование данных на DTO
2	Неправильная обработка статусов HTTP 401/403	Добавлена централизованная обработка ошибок
3	Зависание интерфейса при загрузке данных	Перевод всех операций на асинхронные методы
4	Ошибка подключения к БД при первом запуске	Добавлена автоматическая инициализация БД
5	Дублирование инвентарных номеров	Добавлена проверка уникальности

4.8. Доработка обработкой исключительных ситуаций

В клиентском приложении реализована **централизованная обработка исключений** в классе ApiService.

Обработка различных типов ошибок (Таблица 11):

Таблица 11 – Обработка исключительных ситуаций

№	Код HTTP / Тип ошибки	Сообщение пользователю
1	401 (Unauthorized)	«Неверный логин или пароль, либо недостаточно прав»
2	403 (Forbidden)	«Недостаточно прав для выполнения операции»
3	404 (Not Found)	«Запрашиваемый объект не найден»
4	409 (Conflict)	«Конфликт данных (например, дублирующий инвентарный номер)»

5	500 (Internal Server Error)	«Внутренняя ошибка сервера. Попробуйте позже»
6	Сетевые сбои	«Ошибка соединения с сервером»
7	Невалидные данные	«Проверьте правильность заполнения полей»

Пример обработки ошибки представлен в Листинге 7.

Листинг 7 – Обработка исключений в ApiService

```
private async Task<T> GetAsync<T>(string endpoint)
{
    try
    {
        var response = await _httpClient.GetAsync($"({_baseUrl}{endpoint}");
        var content = await response.Content.ReadAsStringAsync();

        if (!response.IsSuccessStatusCode)
        {
            throw new Exception($"Ошибка API: {response.StatusCode} - {content}");
        }

        return JsonConvert.DeserializeObject<T>(content);
    }
    catch (HttpRequestException)
    {
        MessageBox.Show("Ошибка соединения с сервером", "Ошибка",
            MessageBoxButton.OK, MessageBoxImage.Error);
        throw;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка: {ex.Message}", "Ошибка",
            MessageBoxButton.OK, MessageBoxImage.Error);
        throw;
    }
}
```

4.9. Рефакторинг программного кода

В ходе работы выполнен ряд улучшений (Таблица 12):

Таблица 12 – Выполненные улучшения кода

№	Улучшение	Описание
1	Архитектура клиента	Добавлен слой сервиса ApiService, избавив ViewModel-классы от прямого использования HttpClient
2	Асинхронность	Все команды, связанные с вводом-выводом, переведены на асинхронное выполнение
3	Разделение ответственности	ViewModel-классы теперь отвечают только за логику представления
4	DTO (Data Transfer Objects)	Введены отдельные классы для передачи данных
5	Единообразная обработка ошибок	Все исключения обрабатываются централизованно в ApiService
6	Оптимизация для слабых ПК	Отключены тяжёлые расширения Visual Studio, используется SQL Server Express

4.10. Ручное тестирование

Ручное тестирование проводилось по заранее составленным тест-кейсам, охватывающим функциональность всех ролей. Всего разработано и выполнено **32 теста**.

Фрагмент ручных тест-кейсов представлен в Таблице 13.

Таблица 13 – Фрагмент ручных тест-кейсов

№	ID	Название теста	Шаги	Ожидаемый результат	Статус
1	ТС-01	Вход с корректными данными	1. Ввести admin 2. Ввести password123 3. Нажать «Войти»	Открывается главное окно	Пройден
2	ТС-02	Вход с неверным паролем	1. Ввести admin 2. Ввести wrongpass 3. Нажать «Войти»	Сообщение об ошибке	Пройден

3	ТС-05	Добавление оборудования	1. Нажать «  Оборудование» 2. Заполнить поля 3. Нажать «Сохранить»	Оборудование в списке	Пройден
4	ТС-08	Удаление оборудования (Admin)	1. Выбрать оборудование 2. Нажать «  » 3. Подтвердить	Оборудование удалено	Пройден
5	ТС-12	Попытка удаления (Viewer)	1. Выбрать оборудование 2. Нажать «  »	Кнопка неактивна	Пройден

Результаты ручного тестирования (Таблица 14):

Таблица 14 – Результаты ручного тестирования

№	Роль	Кол-во тестов	Пройдено	Провалено
1	Admin	12	12	0
2	Operator	10	10	0
3	Viewer	7	7	0
4	API	3	3	0
Итого		32	32	0

4.11. Модульное тестирование

Модульное тестирование проведено для клиентского сервиса ApiService с использованием фреймворков **NUnit** и **Moq**.

Результаты модульного тестирования (Таблица 15):

Таблица 15 – Результаты модульного тестирования

№	Тест	Результат
1	GetBuildings_Success_ReturnsBuildings	Пройден
2	GetBuildings_Unauthorized_ThrowsException	Пройден
3	CreateEquipment_Success_ReturnsEquipment	Пройден
4	CreateEquipment_DuplicateInventory_Throws	Пройден
5	DeleteEquipment_Success_NoException	Пройден
6	DeleteEquipment_NotFound_ThrowsException	Пройден
7	NetworkFailure_ThrowsException	Пройден
8	AuthHeaders_AreAddedCorrectly	Пройден
9	GetEquipment_Filtered_Success	Пройден

10	UpdateEquipment_Success_ReturnsUpdated	Пройден
Итого	10 passed, 0 failed	Пройден

Пример модульного теста представлен в Листинге 8.

Листинг 8 – Пример модульного теста

```
[Test]
public async Task GetBuildings_Unauthorized_ThrowsException()
{
    _apiService.SetToken(null);
    var ex = Assert.ThrowsAsync<Exception>(async () =>
        await _apiService.GetBuildingsAsync());
    Assert.That(ex.Message, Does.Contain("401"));
}
```

4.12. Интеграционное тестирование

Интеграционные тесты выполнены для API с помощью **WebApplicationFactory** и **in-memory** базы данных.

Результаты интеграционного тестирования (Таблица 16):

Таблица 16 – Результаты интеграционного тестирования

№	Тест	Результат
1	EquipmentLifecycle_CreateReadUpdateDelete_Success	Пройден
2	BuildingsController_CRUD_Success	Пройден
3	ClassroomsController_CRUD_Success	Пройден
4	EquipmentTypesController_CRUD_Success	Пройден
5	UnauthorizedAccess_Returns401	Пройден
6	ViewerCannotDelete_Returns403	Пройден
7	OperatorCanCreateAndEdit_Success	Пройден
Итого	7 passed, 0 failed	Пройден

Пример интеграционного теста представлен в Листинге 9.

Листинг 9 – Пример интеграционного теста

```
[Test]
public async Task EquipmentLifecycle_CreateReadUpdateDelete_Success()
{
    // ...
}
```

```
// Create
var createData = new { ClassroomId = 1, EquipmentTypeId = 1,
    InventoryNumber = "TEST-001" };
var createResponse = await _client.PostAsync("/api/Equipment",
    JsonContent.Create(createData));
Assert.That(createResponse.StatusCode, Is.EqualTo(HttpStatusCode.Created));

// Read, Update, Delete - аналогично
}
```

4.13. Фиксация результатов в системе контроля версий

Исходный код проекта размещён в системе контроля версий **Git** на платформе **GitHub**. В процессе работы регулярно выполнялись коммиты с осмысленными сообщениями.

Ссылка на репозиторий: https://github.com/Devil-killer987/praktika_pm_02

QR-код для перехода к репозиторию:

Рисунок 4 – QR-код репозитория GitHub



4.14. Разработка руководства оператора

Разработан документ «**Руководство оператора**», описывающий порядок работы с приложением для всех категорий пользователей. Документ содержит инструкции по запуску, входу в систему, основным операциям и действиям в нештатных ситуациях.

Документ размещён в репозитории по
пути: /Файлы/Документы/Руководство_оператора.txt

4.15. Разработка пояснительной записки

Разработана «**Пояснительная записка**», содержащая описание архитектуры системы, перечень реализованных функций, обоснование технических решений и инструкции по развёртыванию.

Документ размещён в репозитории по
пути: /Файлы/Документы/Пояснительная_записка.txt

5. Заключение

В ходе выполнения практики по модулю «Осуществление интеграции программных модулей» была достигнута поставленная цель – разработана информационная система учёта компьютерного оборудования в школе, состоящая из серверного API и клиентского приложения.

Решены следующие задачи (Таблица 17):

Таблица 17 – Выполненные задачи

№	Задача	Результат
1	Анализ предметной области	Построены UML-диаграммы (Use Case) и ER-модель
2	Проектирование БД	Нормализованная БД в 3НФ, 7 таблиц
3	Импорт данных	1000+ записей тестовых данных
4	Разработка API	ASP.NET Core 10, 6 контроллеров, JWT-аутентификация
5	Разработка клиента	WPF .NET Framework 4.8, MVVM, 7 окон
6	Интеграция с API	HTTP-запросы, Bearer Token
7	Отладка и обработка ошибок	Централизованная обработка исключений
8	Рефакторинг	Асинхронные операции, разделение ответственности
9	Тестирование	32 ручных, 10 модульных, 7 интеграционных тестов
10	Документация	Пояснительная записка, руководство оператора
11	Система контроля версий	Git-репозиторий на GitHub

Полученные навыки:

- Работа с Microsoft SQL Server Express в условиях ограниченных ресурсов (6 ГБ ОЗУ)
- Разработка REST API на [ASP.NET](#) Core 10
- Разработка WPF-приложений с использованием MVVM
- Работа с Entity Framework Core

- Реализация аутентификации и авторизации (JWT)
- Интеграция клиент-серверных приложений
- Тестирование программного обеспечения
- Работа с системой контроля версий Git

Благодарности:

Выражаю благодарность руководителю практики от колледжа Седову А.С. и руководителю практики от организации Шешиной Ю.О. за помощь и поддержку.